FREELANCE MARKETPLACE FOR STUDENTS

By

Vishwa Vasoya (23BCA413)

Under Guidance

of

Internal Guide

Dr. Navtej Bhatt

Submitted to



Smt. Chandaben Mohanbhai Patel Institute of Computer Applications
CHARUSAT
Changa

April 2026



Accredited with Grade A+ by NAAC,
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
Changa

Acknowledgement

Knowledge in itself is a continuous process. At this moment of our substantial enhancement, We rarely find words to express our gratitude towards those who were constantly involved with us.

The completion of any inter disciplinary project depends upon coordination, cooperation and combined efforts of several resources of knowledge, creativity, skill, energy and time. The work being accomplished now, we feel our sincerest urge to recall and knowledge through these lines, trying our best to give full credit wherever it deserves.

We would like to thank our project guide **Dr.Navtej Bhatt,** Principal **Dr. Dharmendra Patel** and Dean **Dr. Sanskruti Patel** who advised and gave us moral support through the duration of our project. Without their constant encouragement we could not have been able to achieve what we have.

It's our good fortune that we had support and well wishes of many. We are thankful to all and those names which have been forgotten to acknowledge here but contributions have not gone unnoticed.

With Sincere Regards,

Nit Amipara (23BCA006)

Mahek Shingala (23BCA357)

Vishwa Vasoya (23BCA413)

Table of Contents

Sr. No		Subject	Page Number
1		**Project Profile & Company Profile**	
2		**Introduction to tools**	
3		**System Study**	
	1.	Existing System	
	2.	Proposed System	
	3.	Scope of the Proposed System	
	4.	Aim and Objective of the Proposed System	
	5.	Feasibility Study	
		Operational Feasibility	
		Technical Feasibility	
		Economic Feasibility	
4		**System Analysis**	
		Requirements Specification (along with System Modules)	
		Use Case Diagram	
		Activity Diagram	
		Class Diagram	
		OR	
		System Flowchart Functional Decomposition Diagram DFD ■ Context Level DFD ■ First Level, Second Level ….	
5		**System Design**	
		Data Dictionary	
		Screen Layouts	
		Reports	
6		**System Testing**	
		Testing Strategies	
		Test Cases	
7		**Future Enhancement**	
8		**Bibliography/References**	
9		**Reporting Report (to be collected from respective internal guide)**	

PROJECT PROFILE

❖Project Profile

Project Name: StudentGig – Freelance Marketplace for Students

Type of Application: Web-based Micro-task Recruitment Platform

Project Description: StudentGig is a web application that connects college students with local employers for micro-task based freelance work. Students can browse tasks, submit bid proposals, manage their profiles, track assignments, and earn ratings. Employers can post tasks, review candidates, assign work, and rate completed output. An Admin panel provides platform moderation.

Team Size: 3 Students

Front End: HTML5, CSS3, JavaScript, Chart.js, Font Awesome 6, Google Fonts (Inter)

Back End: PHP (MySQLi – Prepared Statements), Session Management, bcrypt Password Hashing

Tools used: XAMPP (Apache + MySQL), phpMyAdmin, VS Code, Git

INTRODUCTION TO TOOLS

❖ Introduction to Tools

➢ Front End Tool:

HTML5 (HyperText Markup Language)

HTML5 is the standard markup language for creating web pages. In StudentGig, HTML5 is used to structure every page of the application — login forms, dashboards, task listings, profile pages, and admin panels. HTML5 semantic elements such as <header>, <main>, <aside>, and <section> are used to provide meaningful structure and improve accessibility.

CSS3 (Cascading Style Sheets)

CSS3 is used for styling and visual layout of the entire application. A shared assets/css/style.css file defines reusable classes for the sidebar navigation, card components, stat grids, badge indicators, form controls, and responsive media queries. Custom CSS variables (--primary, --bg-light, --border) ensure visual consistency across all portals.

JavaScript

JavaScript handles all client-side interactivity including form validation, dynamic UI feedback, and content toggling. It is also used to pass PHP-generated data (e.g., task status counts) into Chart.js for dynamic chart rendering on the employer dashboard.

Chart.js

Chart.js is an open-source JavaScript charting library integrated into the Employer Dashboard to render a doughnut chart that dynamically shows the distribution of Open, Assigned, and Completed tasks. The chart data is generated server-side in PHP and encoded as JSON for seamless client-side consumption.

Font Awesome 6

Font Awesome 6.4.0 is loaded via CDN and provides scalable vector icon sets used throughout navigation links, dashboard cards, stat indicators, and action buttons, giving the UI a clean and modern look without requiring image assets.

Google Fonts – Inter

The Inter typeface from Google Fonts is used on the login/register pages to deliver a professional, modern typographic style. The sidebar-based dashboards use the same font family for consistency.

➢ Back End Tool:

PHP (Hypertext Preprocessor)

PHP is the server-side scripting language used to power all application logic. Each page in StudentGig is a PHP script that: verifies session-based authentication, enforces role-based access control (candidate/employer/admin), executes prepared MySQLi statements to safely query the database, processes form submissions, manages file redirections, and renders dynamic HTML based on database results.

MySQL via XAMPP

MySQL is the relational database management system storing all application data. The database smart_recruitment contains six tables: users, tasks, applications, reviews, password_resets, and portfolio_items. All tables are linked through foreign keys with CASCADE on delete for referential integrity. XAMPP provides the local Apache + MySQL server environment used during development, with phpMyAdmin as the GUI for database management.

SYSTEM STUDY

Existing System

Currently, students seeking part-time or project-based work rely on general-purpose platforms (Fiverr, Upwork, LinkedIn) or informal channels such as college WhatsApp groups and notice boards. These existing approaches suffer from several critical limitations:

- Generic freelance platforms are not designed for students – they require professional portfolios, charge subscription fees, and are oriented toward full-time freelancers.
- Informal WhatsApp or notice-board systems have no structured proposal or bidding mechanism, making it impossible to compare candidates fairly.
- There is no employer-side task management, applicant shortlisting, or work submission tracking in informal systems.
- No reputation or rating system exists in informal channels, providing no incentive for quality work or reliable delivery.
- Employers have no way to verify a student's skills, educational background, or past work performance.
- Students have no platform to build a digital work portfolio specific to their academic level and capabilities.

Analysis of Existing Platforms

Platform/Method	Strengths	Weaknesses for Students
Fiverr	Global reach, structured gig listings	High competition, 20% commission, complex onboarding, not student-centric
Freelancer.com	Large job pool, bidding system	Professional focus, fees, no leaderboard or academic context
Internshala	India-specific, internship focus	Primarily unpaid or low-paid internships, full-time commitments
LinkedIn Jobs	Professional network	Full-time/part-time jobs only, not suited for micro-tasks
College Notice Boards	Locally relevant	Unstructured, no online application, no rating system, no payment tracking
WhatsApp Groups	Instant communication	No accountability, no formal submission, no reputation system, chaotic

Proposed System

StudentGig is a purpose-built web application that resolves all of the above limitations through a structured, role-based platform. The proposed system introduces the following improvements:

- A dedicated Candidate Portal with dashboard stats, task browsing, proposal submission, application tracking, profile management, and leaderboard access.
- An Employer Portal for posting micro-tasks with budgets and deadlines, reviewing bid proposals, assigning tasks, evaluating submissions, and rating students.
- An Admin Dashboard for platform moderation: banning/unbanning users, deleting tasks, and monitoring overall platform activity.
- A Ratings and Reviews system where employers rate completed work (1–5 stars), building a verifiable reputation for each student.
- A public Leaderboard that ranks the top 20 students by average rating and task completion count, creating healthy competition and recognition.
- Secure authentication with bcrypt password hashing, session-based role enforcement, and a forgot-password OTP flow.

➢ **System Modules**

1) Authentication and User Management Module-

- Multi-role registration: Candidates provide university, skills, bio; Employers provide company/organization name.
- Secure login with bcrypt password verification and role-based redirect (Candidate Dashboard / Employer Dashboard / Admin Panel).
- Forgot Password: User enters registered email; system generates a 6-character alphanumeric code stored in password_resets with a 15-minute expiry; code is emailed to the user; user enters code and new password on the reset page.
- Account suspension: Admin can ban accounts; banned users receive a "Your account has been suspended" message at login.

2) Candidate Module

- Dashboard: Displays real-time statistics (available gigs, applications sent, selections, average rating) and a recent activity table.
- Browse Gigs: Full-text search on task titles, category dropdown filter, responsive card grid with task title, employer name, category badge, price, deadline, and an "Apply" button.
- Apply for Task: Proposal form with bid amount (INR), delivery days, and a cover letter text area. Duplicate applications are prevented.
- My Applications: Table of all submitted applications with task title, bid amount, delivery days, status (Pending / Selected), and a link to submit work if selected.
- Profile: Edit personal information (name, email, location, university, skills), view badge tier (Bronze/Silver/Gold based on completed tasks), view work history with employer reviews.
- Leaderboard: Public ranking of top 20 candidates by average rating (NULL ratings sort to the bottom), with name, skills, task count, and rating stars displayed.

3) Employer Module:-

- Dashboard: Statistics cards (open tasks, ongoing tasks, review-needed, total applicants) and a Chart.js doughnut chart for visual task distribution.
- Post Task: Form to create a task with title, category (8 predefined categories), price (INR, must be > 0), description, deadline, and location (default: Remote).
- Manage Tasks: Table of all posted tasks with status badges, applicant counts, and links to view applicants or review submissions.
- View Applicants: Lists all applicants for a task with name, bid, delivery days, proposal, and a link to the student's public profile. "Assign" button to select a candidate.

- Review Task: Employer reviews submitted work (link + notes), marks task as complete, and submits a 1–5 star rating with optional comment. Duplicate ratings are prevented.

4) Admin Module

- Separate login page (admin_login.php) with hardcoded admin role check.
- Dashboard with platform statistics (total users, tasks, reviews, completed tasks, candidate count, employer count).
- User management table: view all non-admin users, ban/unban toggle, delete user.
- Task management table (last 30 tasks): view with employer name, status, and delete option.

Scope of the Proposed System

The system scope covers the following functional boundaries:

- User registration, login, and logout for all three roles (Student, Employer, Admin).
- Full task lifecycle: Post → Browse → Apply (Bid) → Assign → Submit → Complete → Review.
- Student profile management including education history, bio, skills (technical and soft), and portfolio items.
- Employer task management including task posting, applicant review, assignment, and completion marking.
- Admin moderation: user ban/unban, user deletion, task deletion.
- Leaderboard: Top-20 student ranking by average rating and completed tasks.
- Password recovery via OTP with expiry time management.

Aim and Objective of the Proposed System

Aim

To build a structured, web-based freelance marketplace specifically designed for college students, enabling them to earn income by completing skill-based micro-tasks posted by local employers, while building a verifiable professional reputation.

Objectives

1. To design and implement a role-based web application with distinct, secure dashboards for Students, Employers, and Administrators.
2. To implement a complete task lifecycle from posting through assignment, submission, completion, and rating.
3. To provide students with a digital profile showcasing their education, skills, and portfolio, visible to employers.
4. To enable employers to evaluate candidate suitability through bid proposals, pricing, delivery estimates, and prior ratings before assigning tasks.
5. To motivate students through a transparent public leaderboard ranking them by average rating and tasks completed.
6. To ensure platform security through prepared SQL statements (preventing SQL injection), bcrypt password hashing, and session-based access control.
7. To equip administrators with effective moderation tools to maintain platform integrity.

Feasibility Study

Technical Feasibility

The project uses PHP, MySQL, HTML5, CSS3, and JavaScript – all free and open-source technologies that are taught as part of the BCA curriculum. The development environment, XAMPP, runs on standard consumer-grade hardware. No proprietary or licensed software is required. All team members have the necessary skills to implement and maintain the system. The project is therefore technically feasible.

1.Technology Availability:

All technologies used in StudentGig are freely available, open-source, and widely adopted:

- PHP is pre-installed on virtually all shared hosting environments and XAMPP/WAMP local servers.
- MySQL is the world's most widely deployed open-source RDBMS.
- HTML5, CSS3, and JavaScript are native browser technologies requiring no additional runtime.
- Apache HTTP Server is freely available and bundled with XAMPP.
- Chart.js and Font Awesome are loaded via public CDNs with no cost.

2. Development Environment

- The project can be developed and tested entirely on a local machine using XAMPP, which requires only 400 MB of disk space. Deployment requires any PHP-capable web hosting service (e.g., cPanel hosting, InfinityFree, or a VPS with LAMP stack).

3. Security Feasibility

- The security requirements (SQL injection prevention, XSS prevention, bcrypt hashing, session fixation prevention) are all addressed using native PHP functions — no additional security libraries are required. The implementation complexity is low and well within the capability of a student development team.

4.Technical Feasibility Conclusion

- The project is technically feasible. All required technologies are freely available, well-documented, and appropriate for the scale of this application.

Operational Feasibility

The StudentGig UI is designed with simplicity and intuitiveness in mind. Standard web conventions (sidebar navigation, card layouts, form-based submissions, status badges) ensure that no technical training is required for any user – student, employer, or administrator. The platform is accessible from any device with a modern web browser. The system is operationally feasible.

1.Operational Feasibility:

Operational feasibility determines whether the system can be successfully used by the intended audience in its operational environment.

- **Ease of Use**
  - The user interface uses familiar web conventions (forms, buttons, cards, tables) that match the mental model of students and employers who use everyday web applications.
  - Role-based navigation ensures users only see menus and pages relevant to their role, reducing cognitive load.

- Inline error messages (e.g., "Invalid password", "No account found with that email") guide users through authentication without confusion.

➢ **Training Requirements**

- No formal training is required. The UI is self-explanatory with labelled navigation, clear CTAs (Call-to-Action buttons), and descriptive page headings.
- A brief onboarding guide or FAQ page (future enhancement) would be sufficient for any edge cases.

➢ **System Maintenance**

- The admin panel provides a no-code interface for content moderation (ban users, delete tasks) without requiring database access.
- The codebase is structured by feature (one PHP file per feature: post_task.php, browse_tasks.php, etc.) making maintenance and updates straightforward.

➢ **Operational Feasibility Conclusion**

- The system is operationally feasible. It uses intuitive web UI patterns, requires no training, and is maintainable by a non-technical administrator through the admin panel.

Economical Feasibility

The entire technology stack is free and open-source. Development hardware is existing college infrastructure. Deployment can be achieved on low-cost shared hosting with no licensing fees. The cost-benefit analysis clearly favours development, as the platform delivers significant academic and practical value at near-zero cost. The project is economically feasible.

1.Cost Analysis:

The project is economically feasible. Development cost is near-zero using entirely free and open-source tools. Deployment cost is minimal and optional for academic use.

Cost Item	Cost (INR)	Notes
PHP, MySQL, Apache (XAMPP)	0	Free and open-source
VS Code IDE	0	Free
Font Awesome (CDN)	0	Free tier
Google Fonts	0	Free
Chart.js (CDN)	0	Free and open-source
Web Hosting (optional deployment)	1,500 – 3,000/year	Shared PHP hosting if deployed live
Domain Name (optional)	800 – 1,200/year	e.g., .in or .com domain

Development Labour	Academic (nil)	Built as academic project
TOTAL	≈ 0 – 4,200 / year	Minimal cost for full-featured platform

SYSTEM ANALYSIS

❖ Requirements Specification:

1. Functional Requirements:

- **Authentication Requirements**

Req ID	Requirement	Actor	Priority
FR-01	System shall allow users to register as Candidate or Employer with email and password	All Users	High
FR-02	System shall authenticate users using email/password and route by role to appropriate dashboard	All Users	High
FR-03	System shall prevent login for accounts with is_banned = 1	System	High
FR-04	System shall provide a forgot-password flow: email entry → code sent → code verified → new password set	Candidate/Employer	Medium
FR-05	System shall regenerate session ID on successful login	System	High

- **Candidate Requirements**

Req ID	Requirement	Priority
FR-06	**Candidate shall view a dashboard with stats: available gigs, applications, selections, avg. rating**	**High**
FR-07	**Candidate shall browse open tasks with search (title keyword) and category filter**	**High**
FR-08	**Candidate shall submit a task application with**	**High**

	bid amount, delivery days, and proposal text	
FR-09	System shall prevent a candidate from applying	High

Req ID	Requirement	Priority
FR-10	Candidate shall view all applications and their status (Pending / Selected)	High
FR-11	Selected candidate shall submit work via a submission link and note	High
FR-12	Candidate shall edit profile: name, email, location, university, skills, bio	Medium
FR-13	Candidate shall view their badge tier (Bronze/Silver/Gold) based on completed tasks	Medium
FR-14	Candidate shall view a public leaderboard of top 20 freelancers by avg. rating	Medium

- **Employer Requirements**

FR-15	**Employer shall view a dashboard with stats: open/ongoing/completed tasks, total applicants, chart**	**High**
FR-16	**Employer shall post tasks with title, category, price (> 0), description, deadline**	**High**
FR-17	**Employer shall view all applicants for a task with their bid, proposal, and profile link**	**High**
FR-18	**Employer shall assign exactly one candidate per task; task status changes to "assigned"**	**High**
FR-19	**Employer shall review submitted work and mark task as "completed"; task status updates**	**High**
FR-20	**Employer shall rate the completed work (1–5 stars) with optional comment (once per task)**	**High**

- **Admin Requirements**

Req ID	Requirement	Priority
FR-21	**Admin shall login via a dedicated admin login page**	**High**
FR-22	**Admin shall view platform statistics: total users, tasks, reviews, completed tasks, role counts**	**High**
FR-23	**Admin shall ban any non-admin user; banned user cannot login**	**High**
FR-24	**Admin shall unban a previously banned user**	**High**
FR-25	**Admin shall delete any non-admin user (CASCADE deletes their tasks and applications)**	**High**
FR-26	**Admin shall delete any task from the platform**	**Medium**

2.Non-Functional Requirements:

Req ID	Category	Requirement

NFR-01	Security	All SQL queries shall use prepared statements to prevent SQL injection
NFR-02	Security	Passwords shall be hashed using bcrypt (PHP password_hash / password_verify)
NFR-03	Security	All user-supplied output in HTML shall be escaped using htmlspecialchars()
NFR-04	Security	Session IDs shall be regenerated on each login (session_regenerate_id)
NFR-05	Performance	Dashboard statistics queries shall execute within 500ms on standard hardware
NFR-06	Usability	All pages shall be functional on Chrome, Firefox, Edge (latest versions)
NFR-07	Usability	The landing page and browse page shall be responsive on screens ≥ 320px wide
NFR-08	Reliability	Session data shall persist across page navigations within a single browser session
NFR-09	Maintainability	Each feature shall be implemented in a dedicated PHP file for ease of maintenance
NFR-10	Data Integrity	All foreign keys shall have ON DELETE CASCADE to prevent orphaned records
NFR-11	Accessibility	All form inputs shall have associated labels and placeholder text
NFR-12	Availability	Platform shall be accessible 24/7 on any Apache-PHP server with MySQL support

System Modules

The StudentGig system is divided into the following major modules:

- Authentication Module: Registration, Login, Logout, Forgot Password, OTP Reset (register.html, register.php, login.php, logout.php, forgot_password.php, reset_password.php).
- Candidate Module: Dashboard, Browse Tasks, Apply/Bid, My Applications, Submit Work, Profile, Leaderboard (candidate_dashboard.php, browse_tasks.php, apply_logic.php, my_applications.php, submit_task.php, profile.php, leaderboard.php).
- Employer Module: Dashboard, Post Task, View Applicants, Assign Task, My Tasks/Gigs, Review & Complete Task (employer_dashboard.php, post_task.php, view_applicants.php, assign_task.php, my_gigs.php, review_task.php).
- Admin Module: Admin Login, Admin Dashboard with user/task moderation (admin_login.php, admin_dashboard.php).
- Database Module: Centralised db.php for MySQL connection; setup.sql for full schema initialisation.

Use Case Diagram

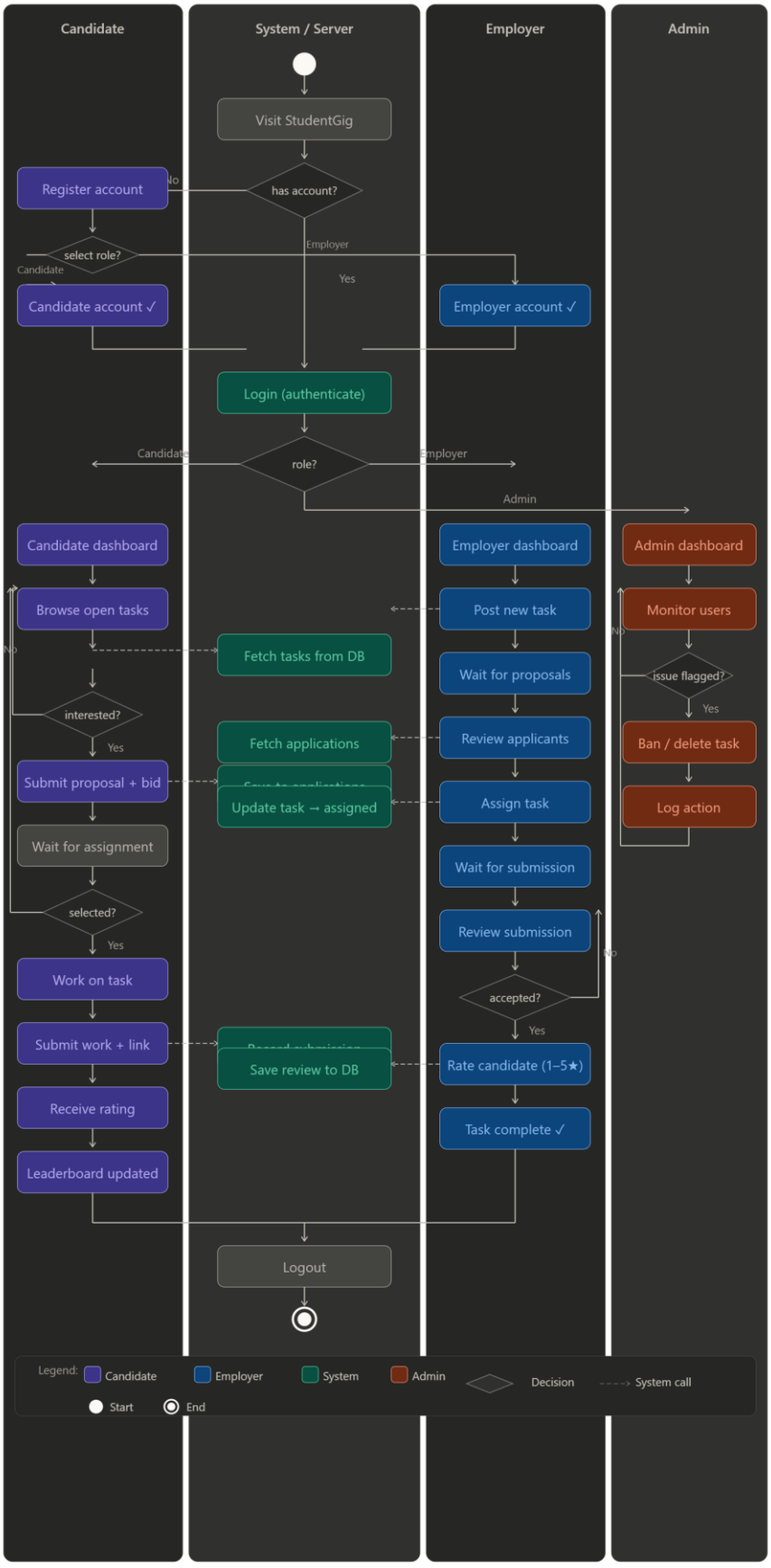


Activity Diagram

The primary activity flow for the Task Lifecycle in StudentGig is as follows:

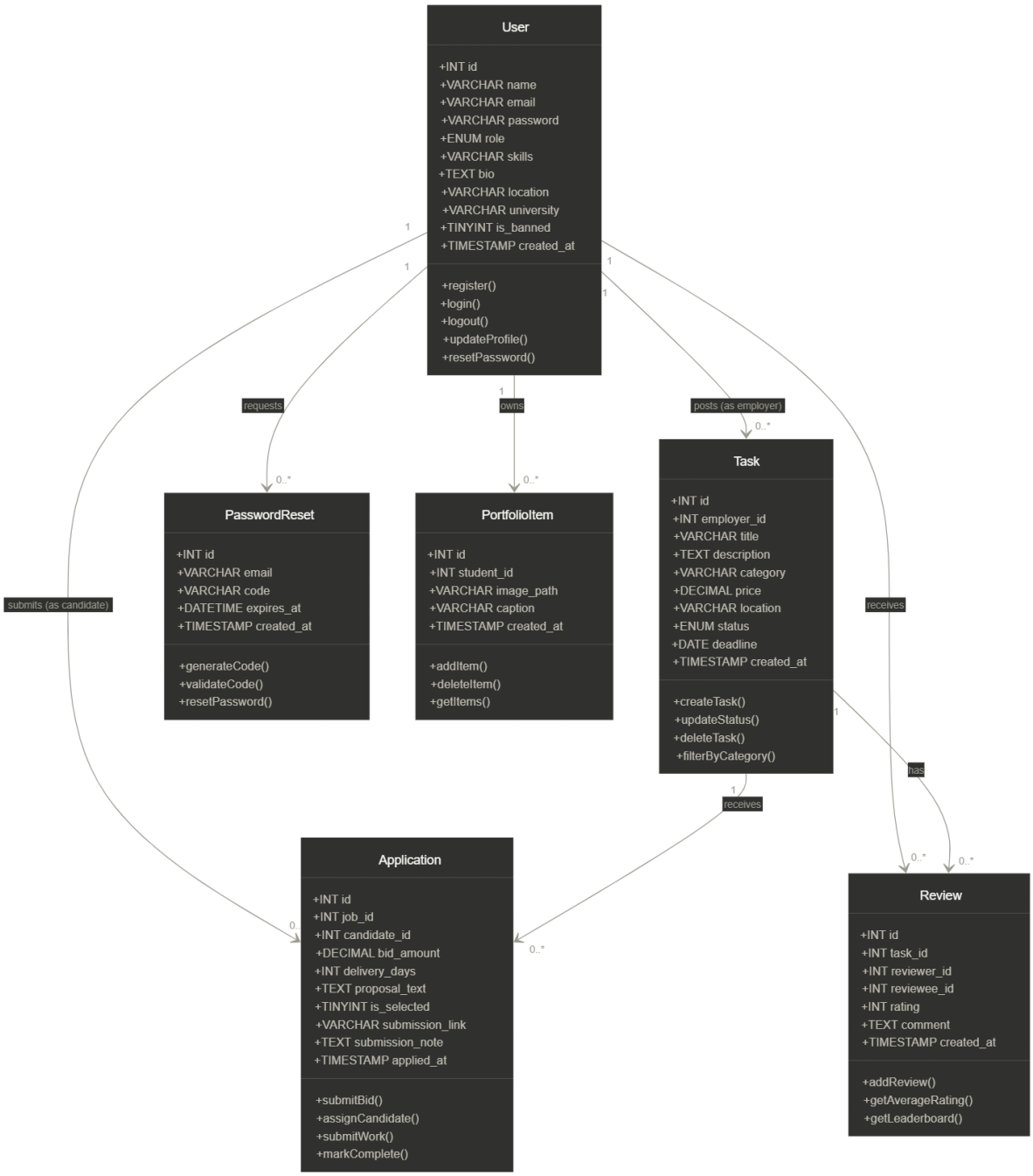
- Start → Employer logs in → Posts a Task (status = open).
- Student logs in → Browses open tasks → Submits a Bid Proposal.
- Employer reviews proposals → Selects a candidate (is_selected = 1; task status = assigned).
- Student receives assignment → Completes the work → Submits work link via submit_task.php.
- Employer reviews submission → Approves and marks task Complete (status = completed).
- Employer submits a Rating (1–5 stars) → Review stored → Leaderboard rank updated → End.

If the employer rejects the submission, the student is notified and may resubmit. If no candidate is suitable, the employer may delete the task or leave it open.

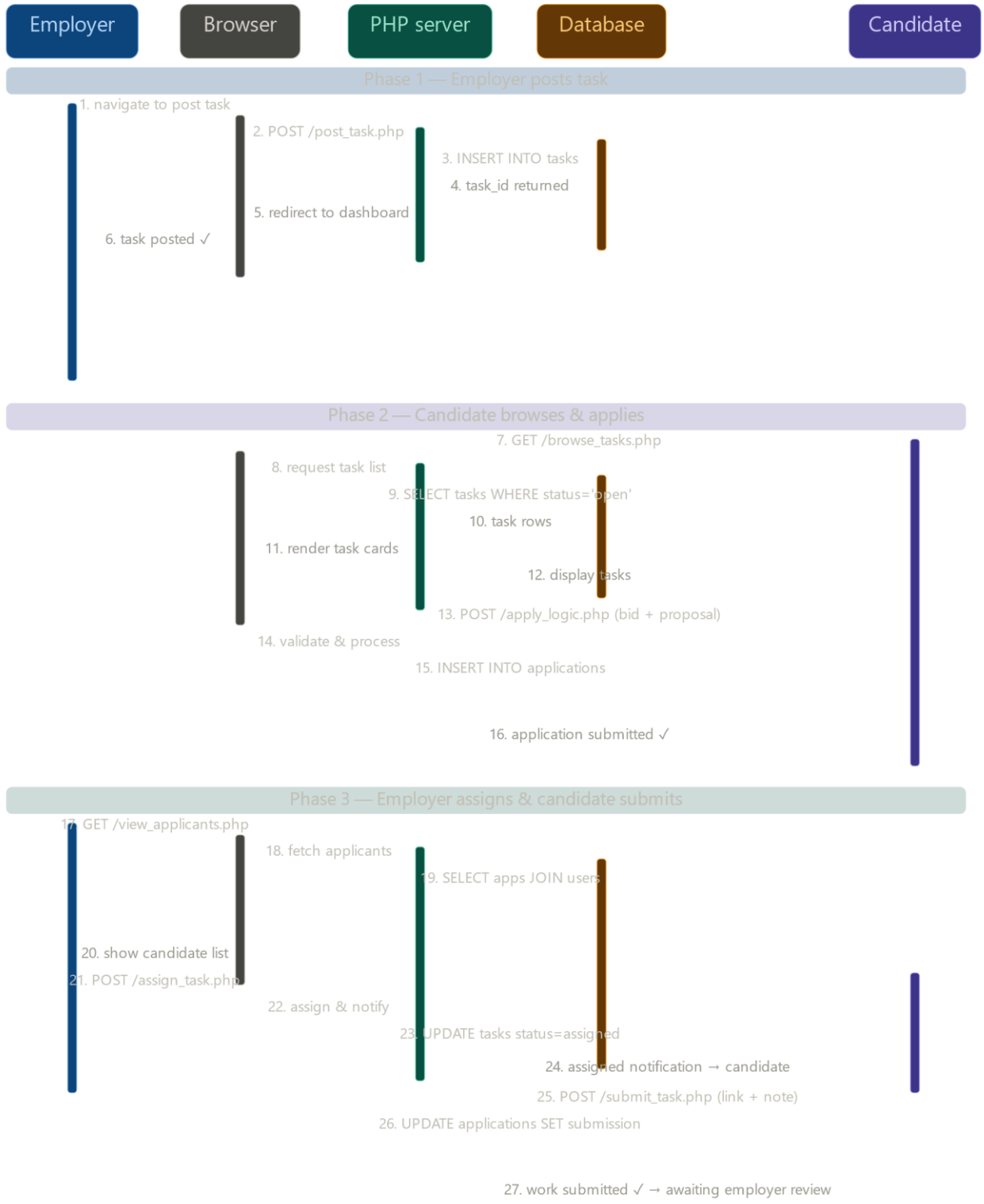

Candidate
System / Server
Employer
Admin
Visit StudentGig
Register account
has account?
No
select role?
Candidate
Employer
Yes
Candidate account ✓
Employer account ✓
Login (authenticate)
role?
Candidate
Employer
Admin
Candidate dashboard
Employer dashboard
Admin dashboard
Browse open tasks
Post new task
Monitor users
No
Fetch tasks from DB
Wait for proposals
issue flagged?
Yes
interested?
Yes
Fetch applications
Review applicants
Ban / delete task
Submit proposal + bid
Update task → assigned
Assign task
Log action
Wait for assignment
Wait for submission
selected?
Yes
Review submission
No
Work on task
accepted?
Yes
Submit work + link
Save review to DB
Rate candidate (1–5★)
Receive rating
Task complete ✓
Leaderboard updated
Logout
Legend:
Candidate
Employer
System
Admin
Decision
System call
Start
End

Class Diagram

The object-oriented model of StudentGig includes the following principal classes:

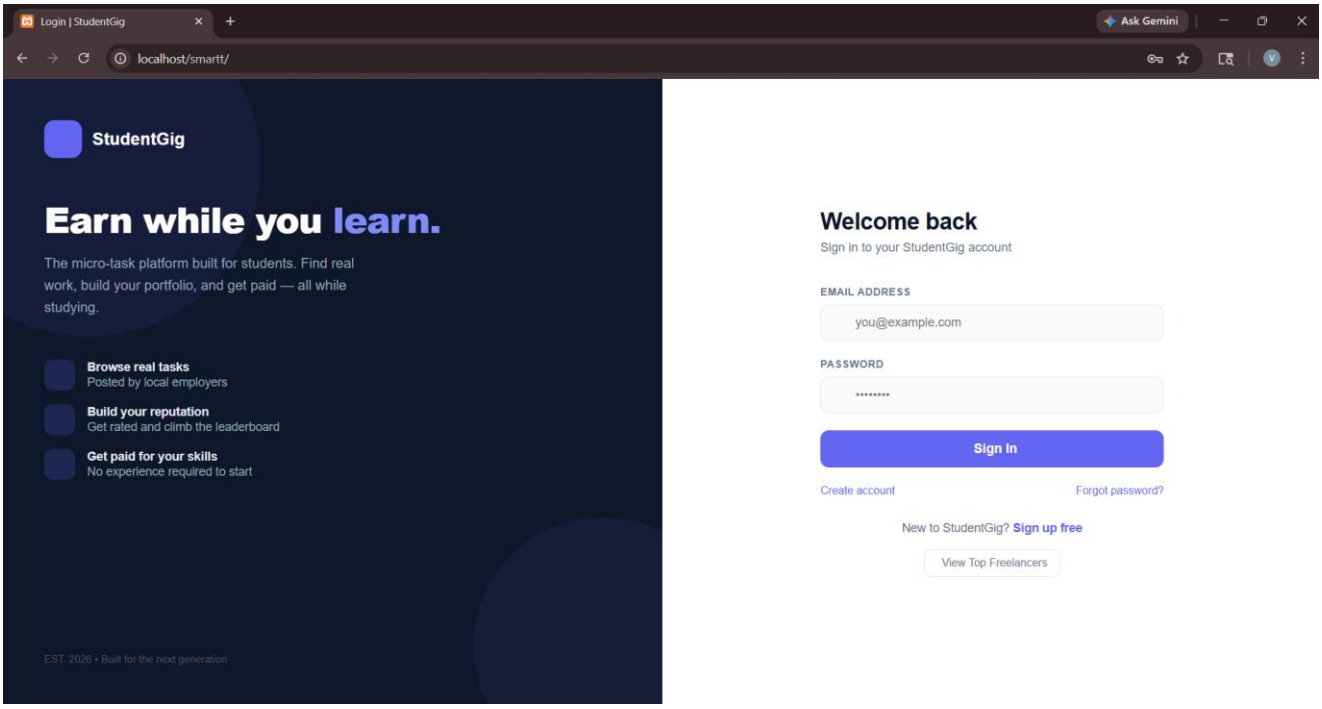


Sequence Diagram

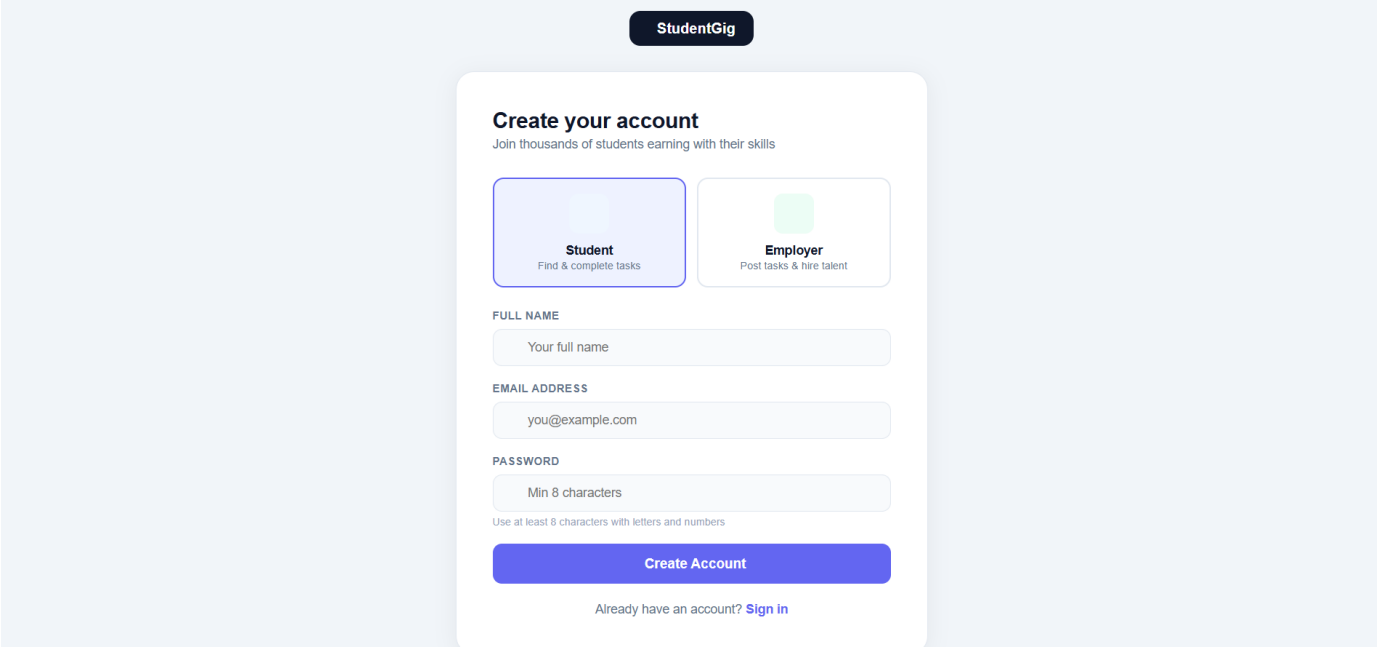


SYSTEM DESIGN

LOGIN PAGE:

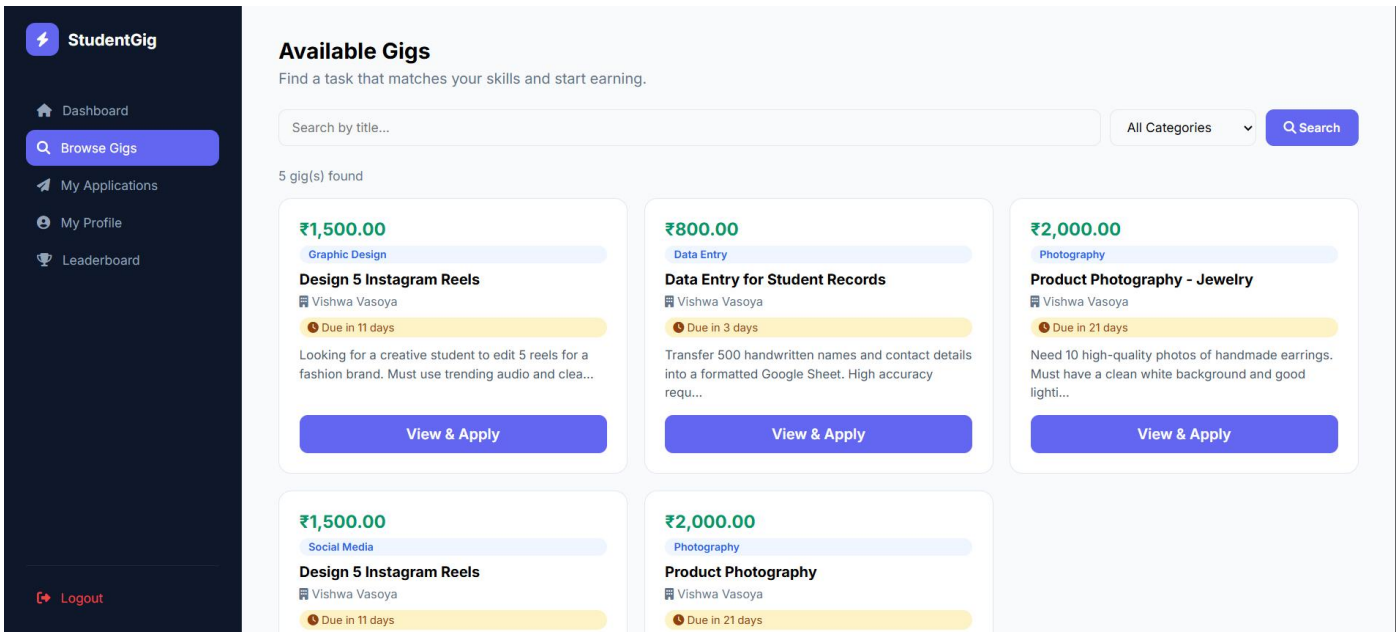


SIGN UP PAGE:

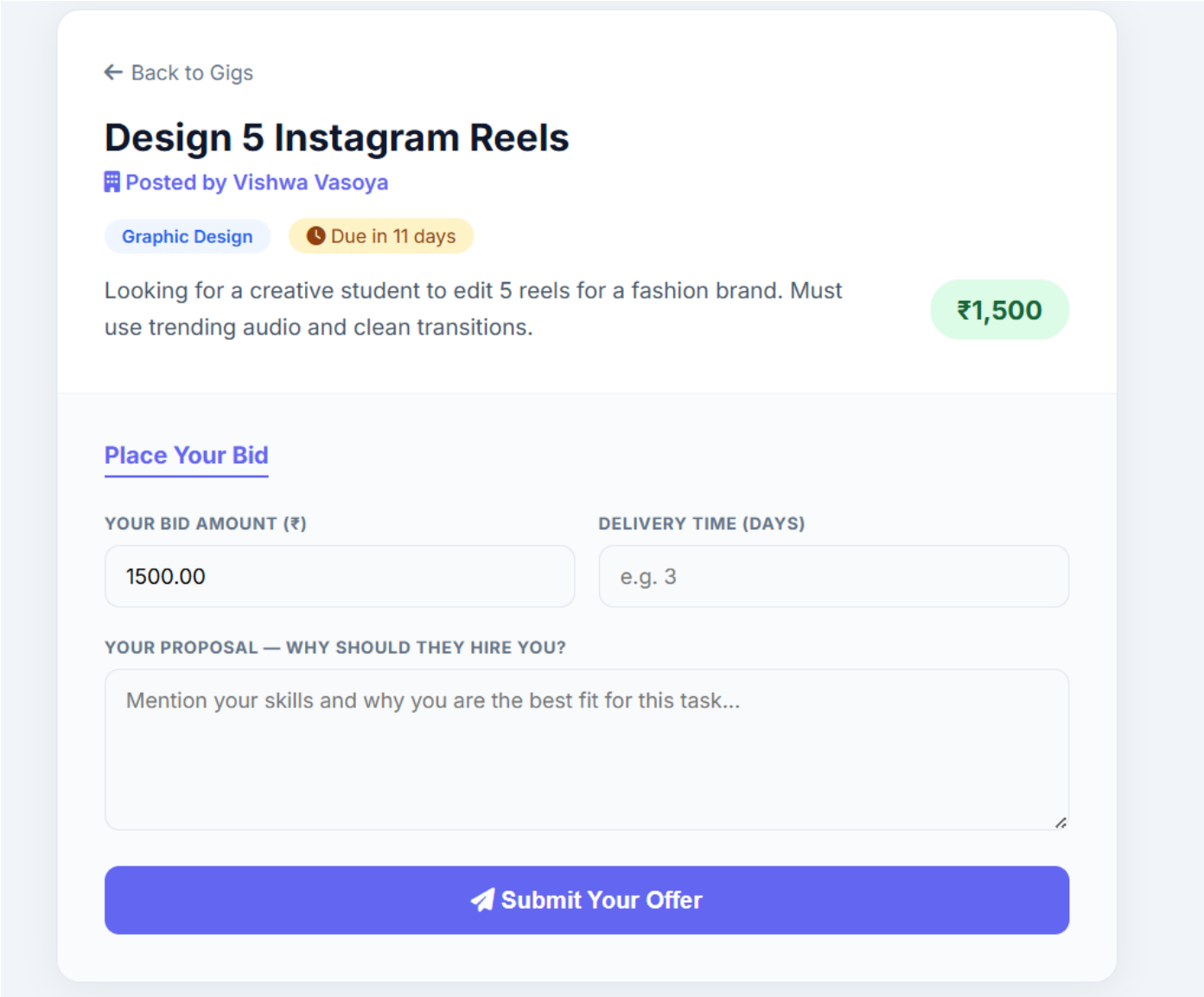


User:

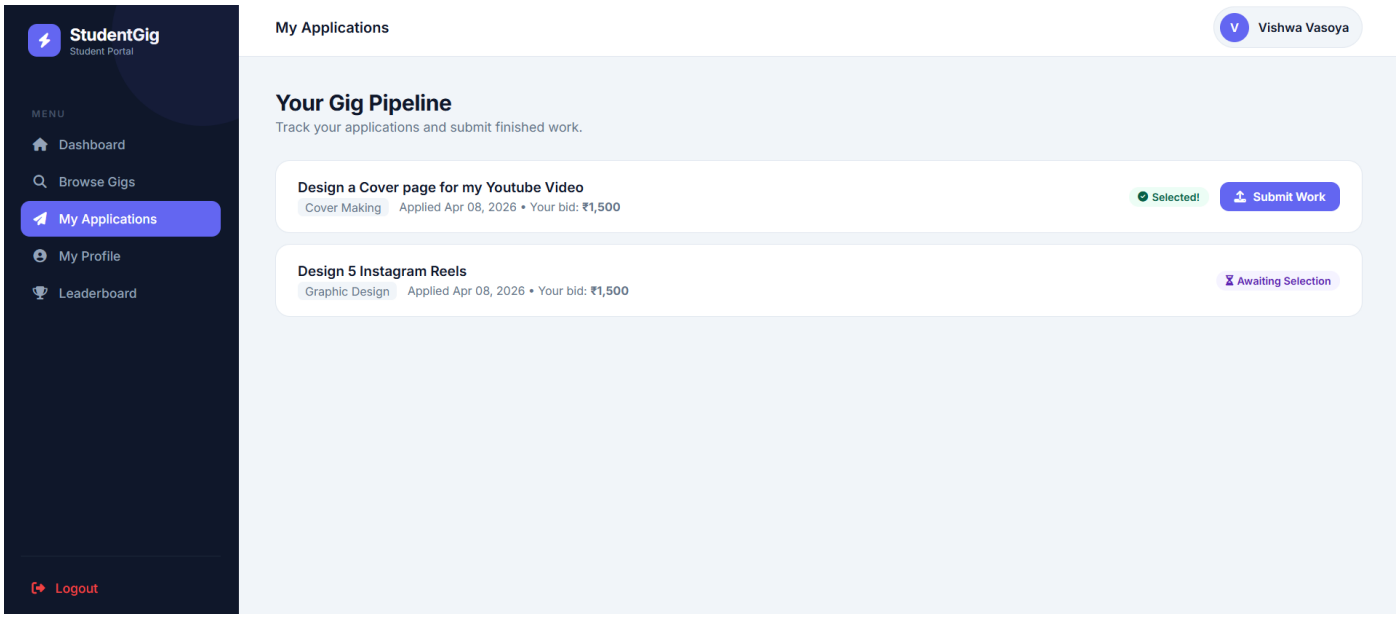
HOME PAGE:



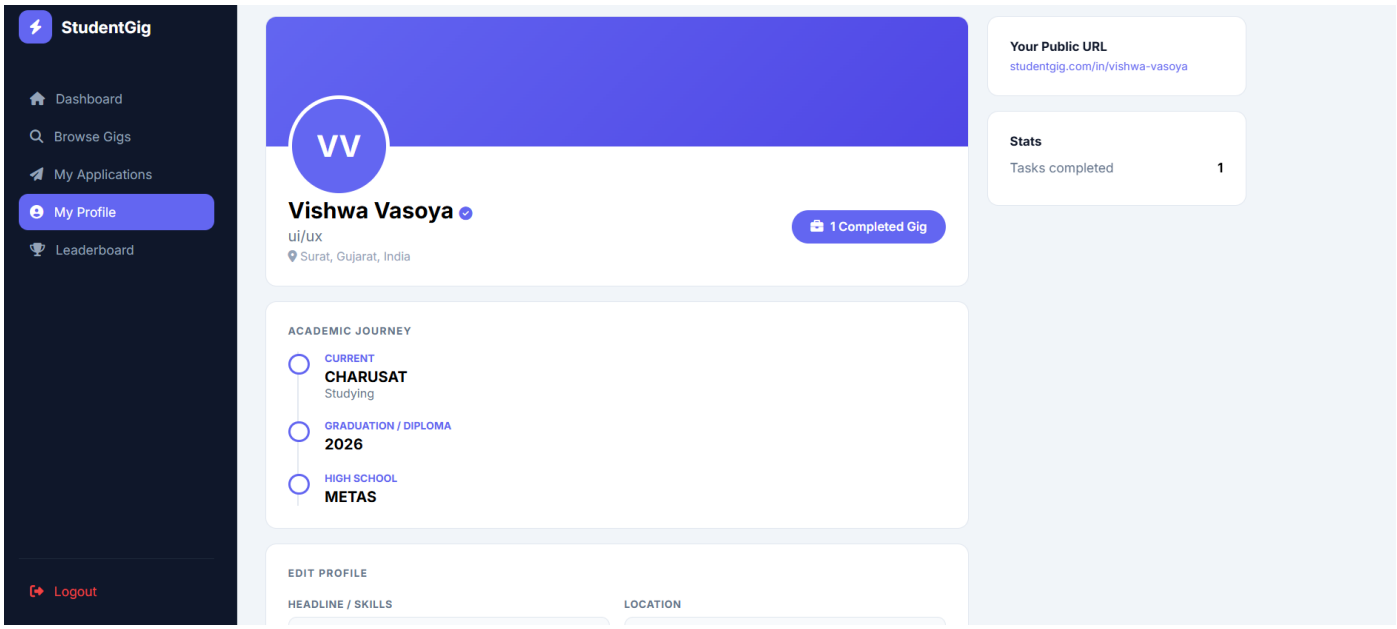
BID PAGE:

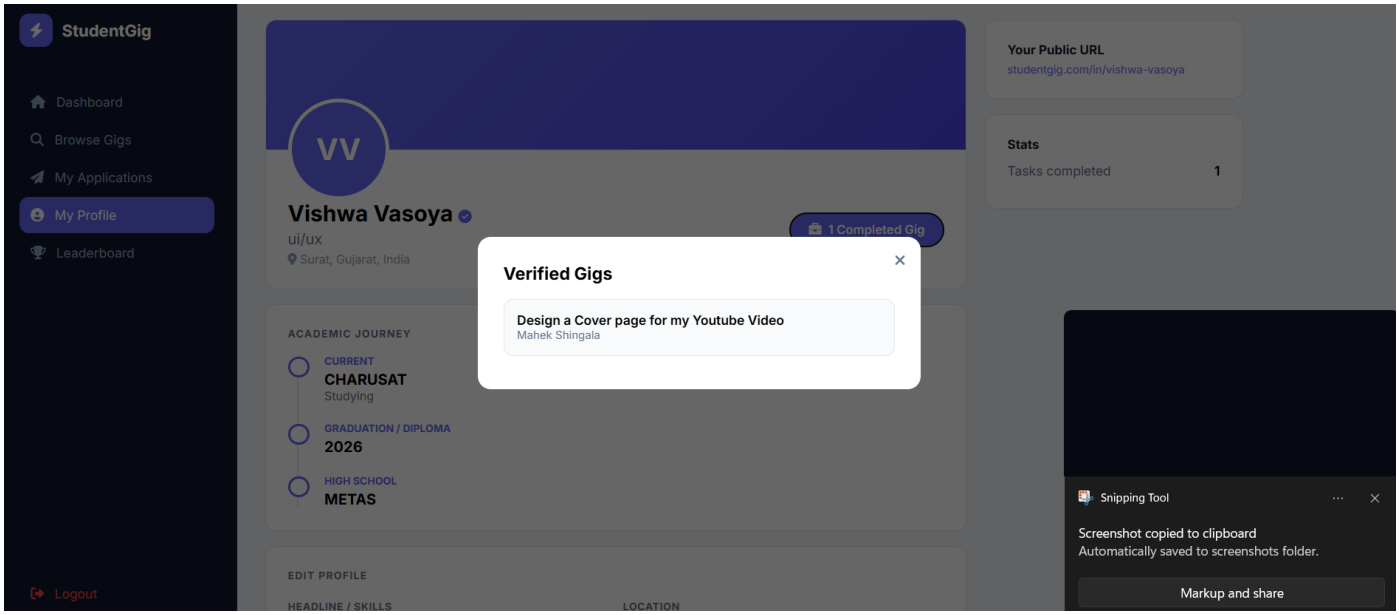

Back to Gigs
Design 5 Instagram Reels
Posted by Vishwa Vasoya
Graphic Design
Due in 11 days
Looking for a creative student to edit 5 reels for a fashion brand. Must use trending audio and clean transitions.
₹1,500
Place Your Bid
YOUR BID AMOUNT (₹)
1500.00
DELIVERY TIME (DAYS)
e.g. 3
YOUR PROPOSAL — WHY SHOULD THEY HIRE YOU?
Mention your skills and why you are the best fit for this task...
Submit Your Offer

MY APPLICATION PAGE:



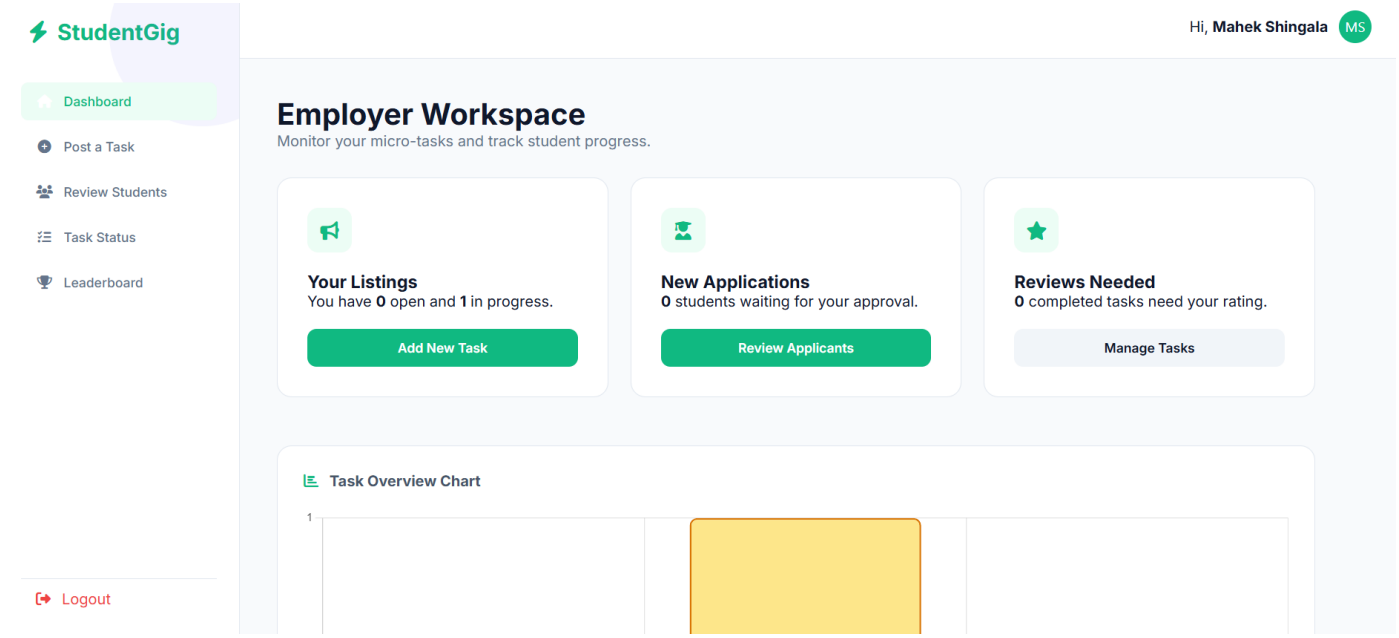
PROFILE PAGE:

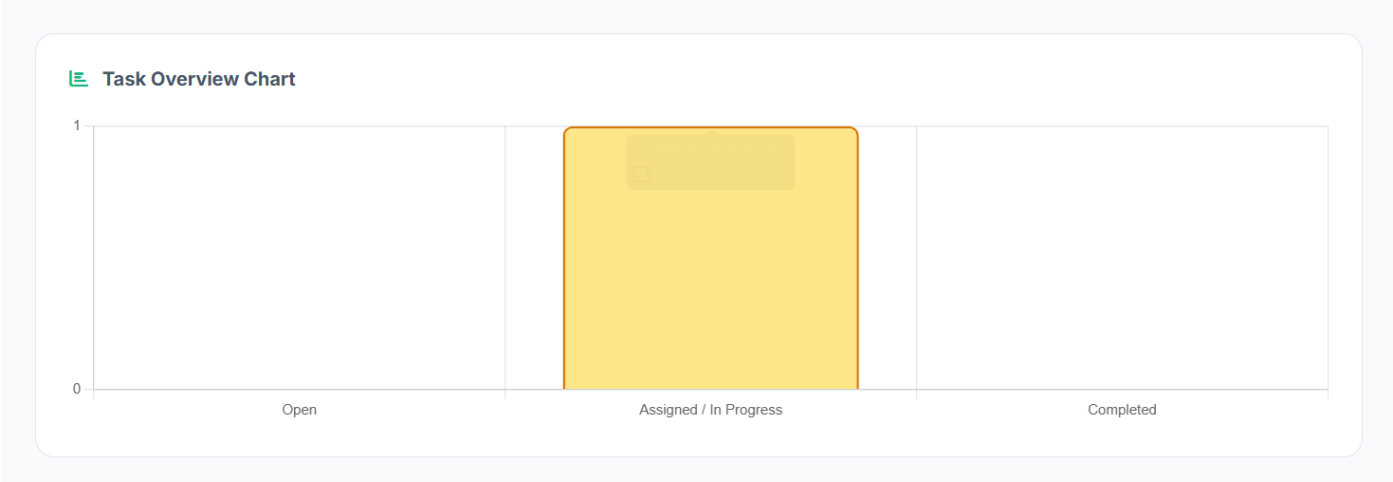




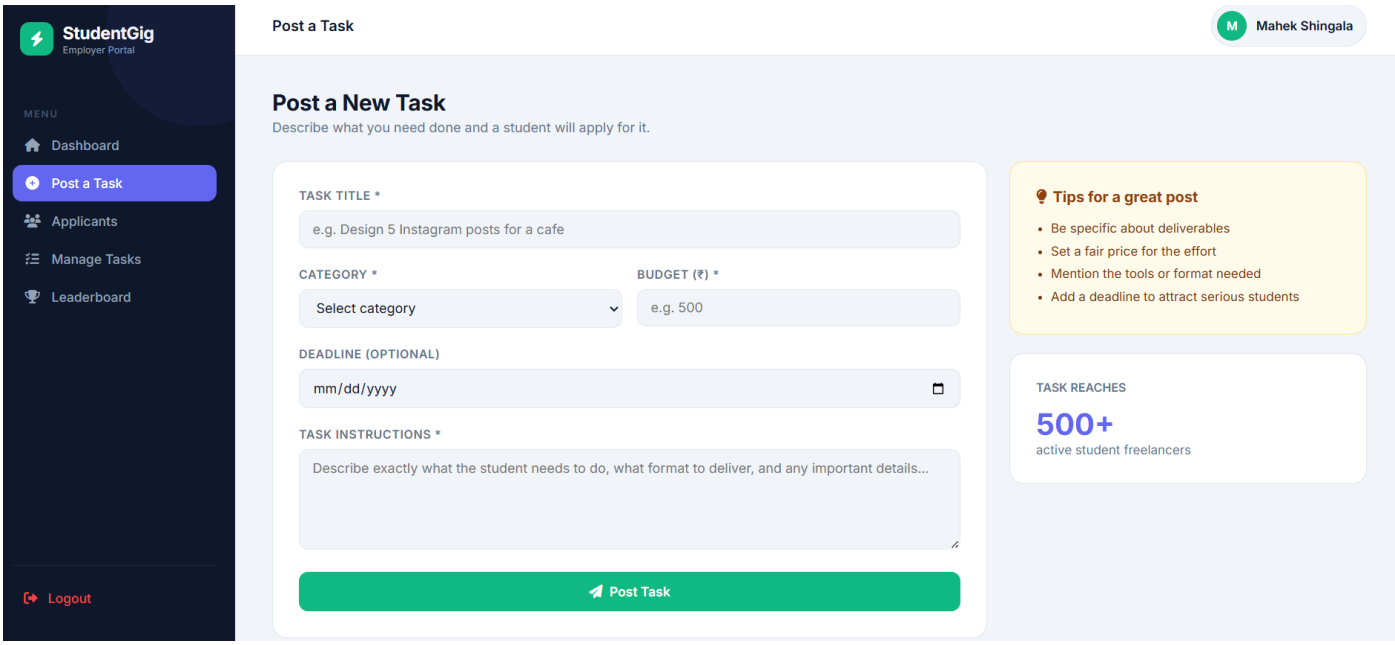
Employer:

HOME PAGE:

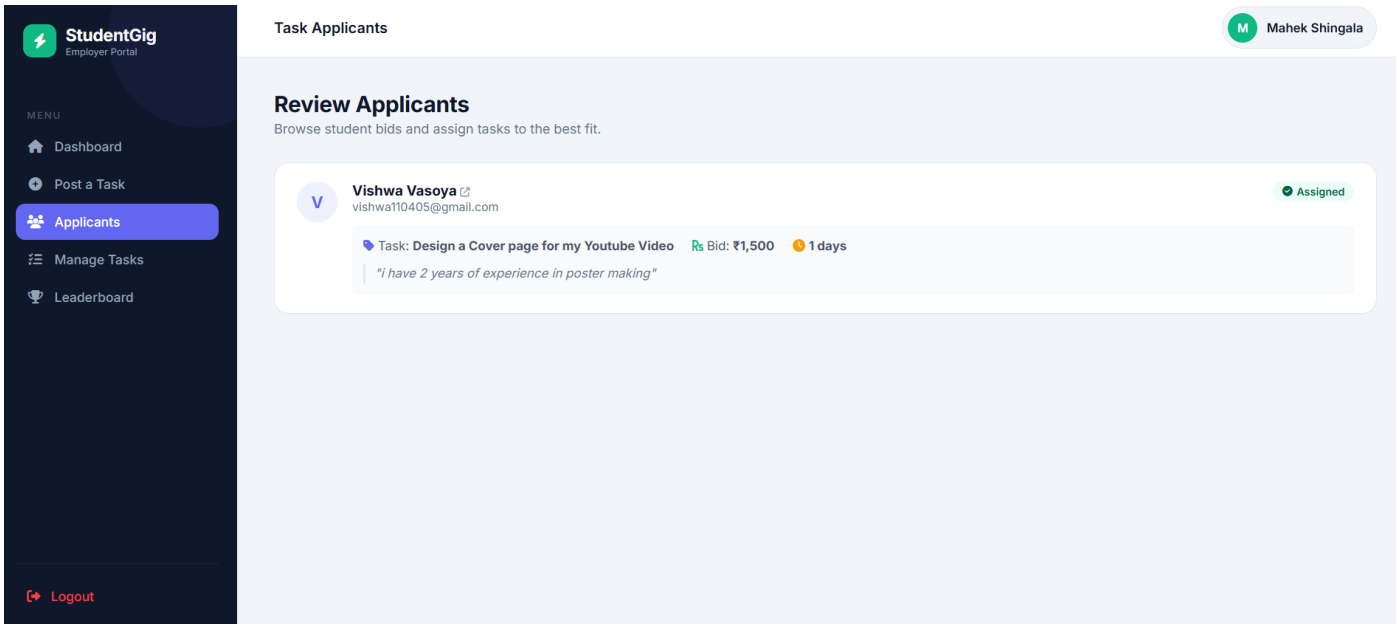




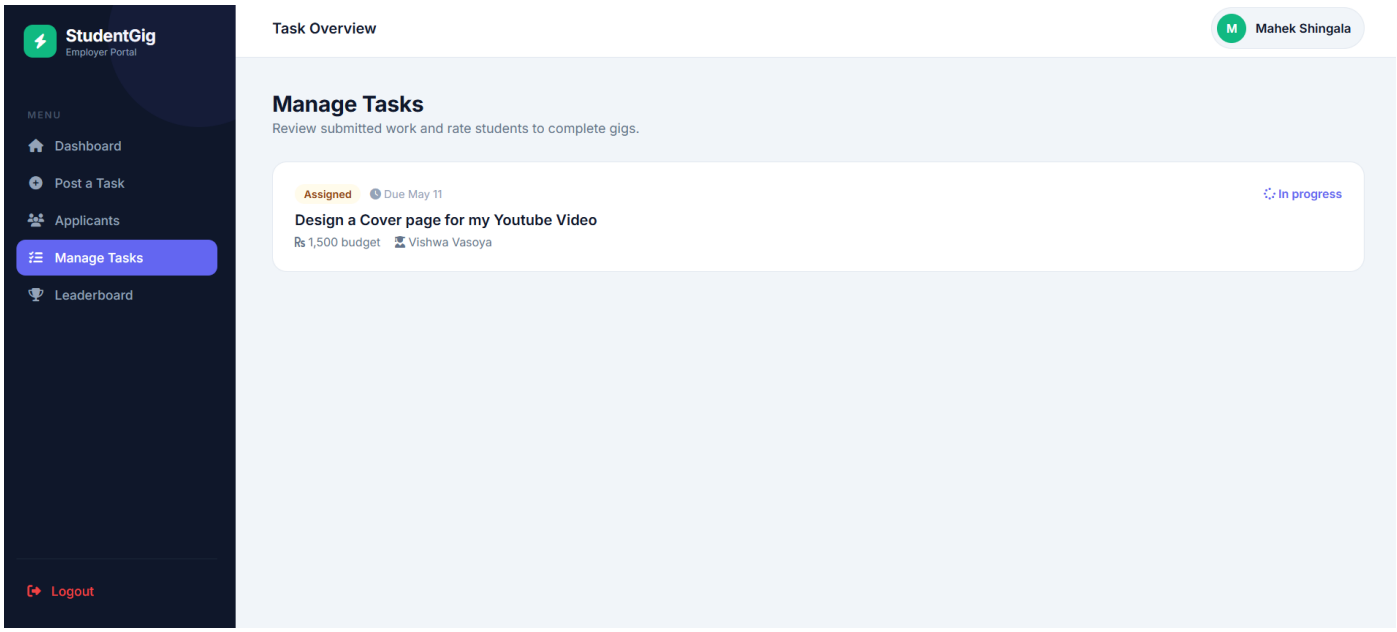
POST TASK PAGE:



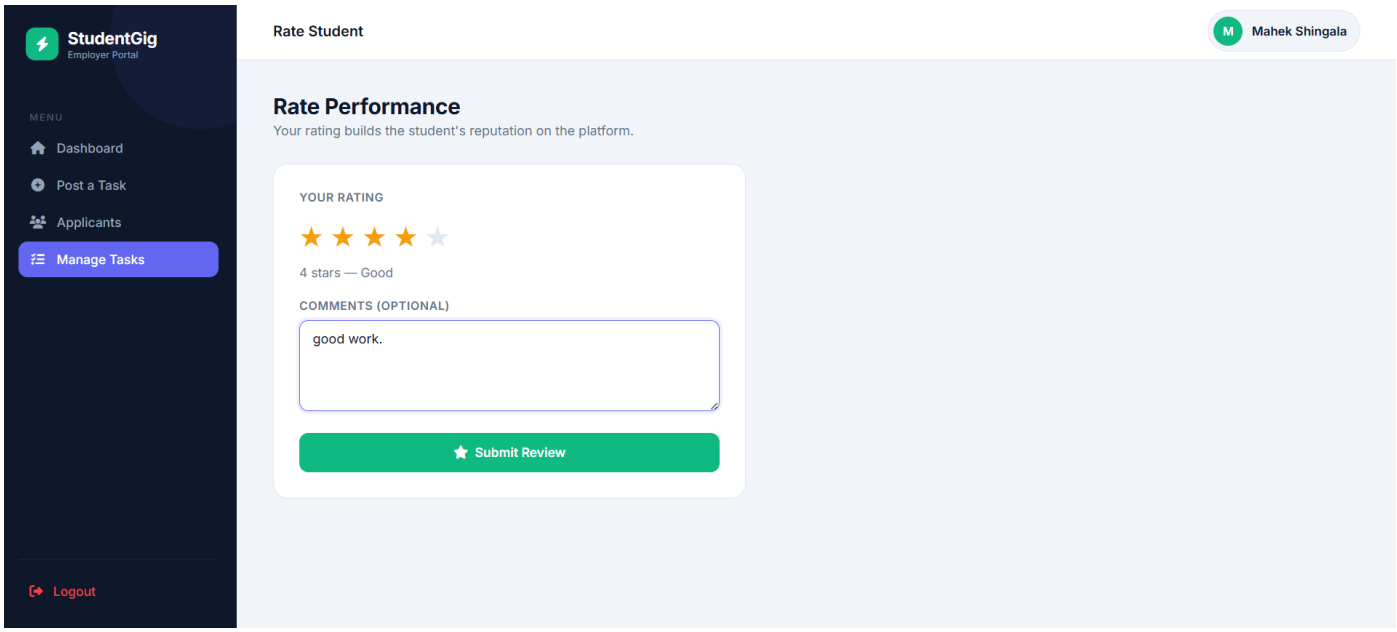
APPLICANTS PAGE:



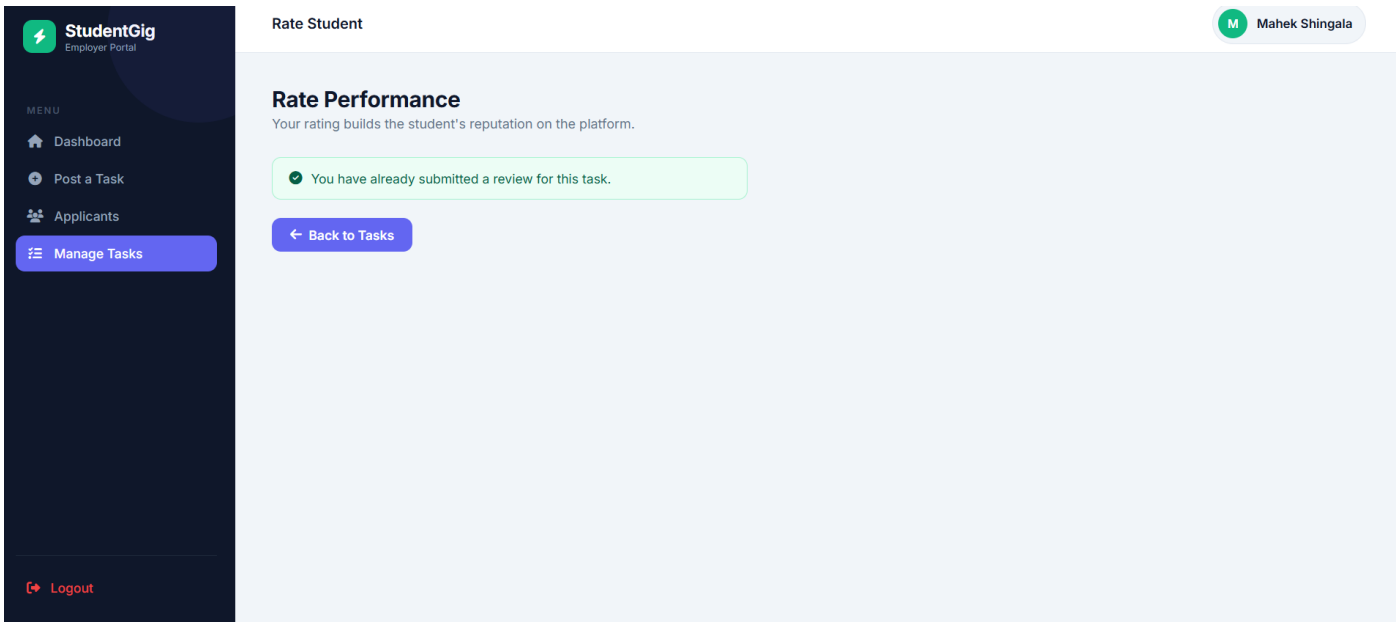
MANAGE TASKS PAGE:



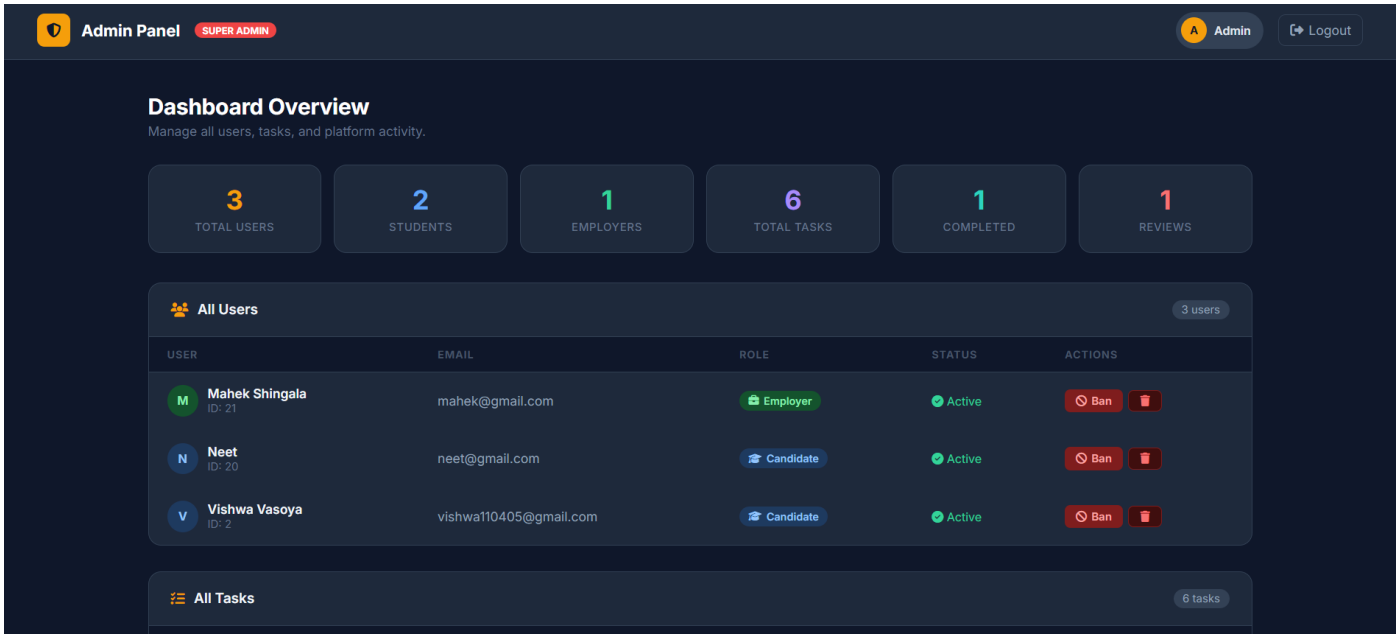
MANAGE TASK PAGE:

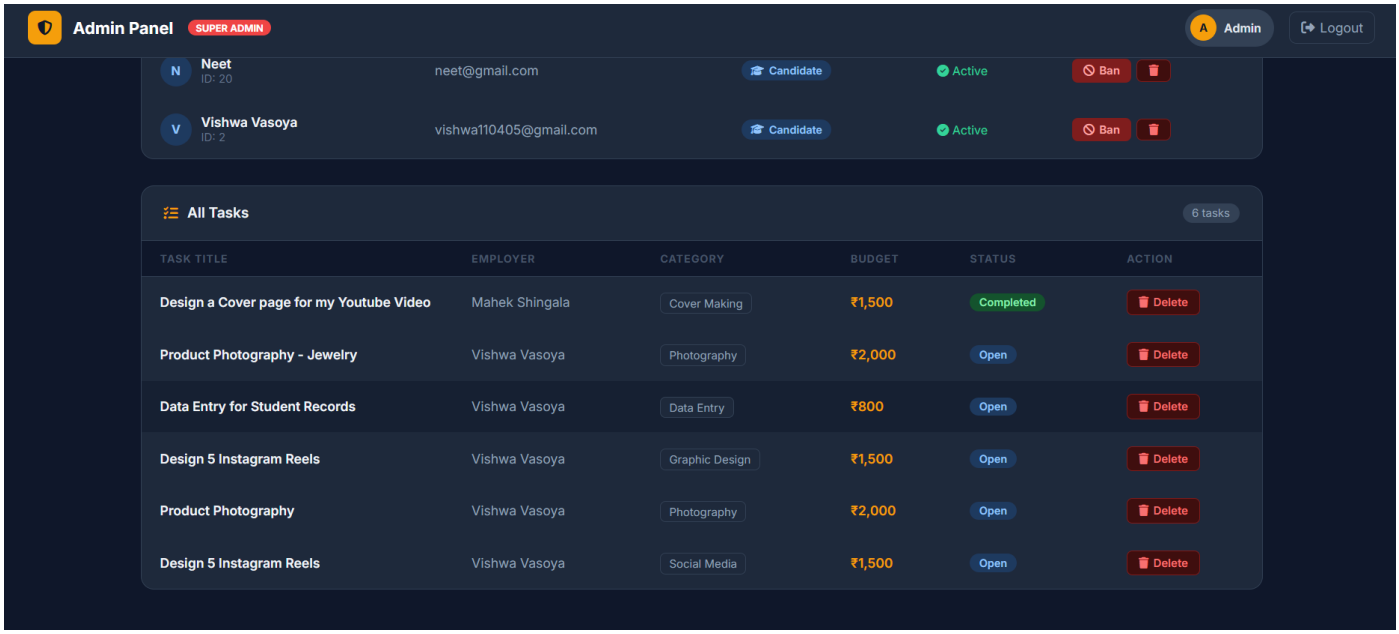


REVIEW PAGE:



ADMIN:


Admin Panel
SUPER ADMIN
Admin
Logout
Dashboard Overview
Manage all users, tasks, and platform activity.
3
TOTAL USERS
2
STUDENTS
1
EMPLOYERS
6
TOTAL TASKS
1
COMPLETED
1
REVIEWS
All Users
3 users
USER
EMAIL
ROLE
STATUS
ACTIONS
Mahek Shingala
ID: 21
mahek@gmail.com
Employer
Active
Ban
Neet
ID: 20
neet@gmail.com
Candidate
Active
Ban
Vishwa Vasoya
ID: 2
vishwa110405@gmail.com
Candidate
Active
Ban
All Tasks
6 tasks


Admin Panel
SUPER ADMIN
Admin
Logout
Neet
ID: 20
neet@gmail.com
Candidate
Active
Ban
Vishwa Vasoya
ID: 2
vishwa110405@gmail.com
Candidate
Active
Ban
All Tasks
6 tasks
TASK TITLE
EMPLOYER
CATEGORY
BUDGET
STATUS
ACTION
Design a Cover page for my Youtube Video
Mahek Shingala
Cover Making
₹1,500
Completed
Delete
Product Photography - Jewelry
Vishwa Vasoya
Photography
₹2,000
Open
Delete
Data Entry for Student Records
Vishwa Vasoya
Data Entry
₹800
Open
Delete
Design 5 Instagram Reels
Vishwa Vasoya
Graphic Design
₹1,500
Open
Delete
Product Photography
Vishwa Vasoya
Photography
₹2,000
Open
Delete
Design 5 Instagram Reels
Vishwa Vasoya
Social Media
₹1,500
Open
Delete

SYSTEM TESTING

Testing Strategy

The testing strategy for StudentGig is composed of four levels:

- Unit Testing: Each PHP function/module is tested in isolation. For example, the login function is tested with valid credentials, invalid password, non-existent email, and banned account scenarios.
- Integration Testing: Combined module interactions are tested. For example, the Apply → Assign → Submit → Review flow is tested end-to-end across multiple database tables.
- System Testing: The complete web application is tested against all functional and non-functional requirements.
- User Acceptance Testing (UAT): Representative users (students and employers) use the platform and verify it meets their expectations.

Test Cases – Authentication Module

TC ID	Test Case	Input	Expected Output	Result
TC-01	Valid login – Candidate	Registered candidate email + correct password	Redirect to candidate_dashboard.php	Pass
TC-02	Valid login – Employer	Registered employer email + correct password	Redirect to employer_dashboard.php	Pass
TC-03	Valid login – Admin	admin@studentgig.com + Admin@1234	Redirect to admin_dashboard.php	Pass
TC-04	Invalid password	Registered email + wrong password	Error: "Invalid password."	Pass
TC-05	Non-existent email	unregistered@x.com + any password	Error: "No account found with that email."	Pass
TC-06	Banned account login	Banned user credentials	Error: "Your account has been suspended."	Pass
TC-07	Forgot password – valid email	Registered email in forgot_password.php	Code sent; redirected to reset page	Pass
TC-08	Reset with valid code	Correct code within 15 minutes + new password	Password updated; login with new password works	Pass
TC-09	Reset with expired code	Code used after 15 minutes	Error: "Code expired or invalid."	Pass
TC-10	Registration with duplicate email	Email already in users table	Error displayed; no new record created	Pass

Test Cases – Candidate Module

TC ID	Test Case	Input	Expected Output	Result
TC-11	Browse all open tasks	Load browse_tasks.php (logged in as candidate)	All open tasks displayed in card grid	Pass
TC-12	Search task by keyword	Search = "data"	Only tasks with "data" in title shown	Pass
TC-13	Filter by category	Category = "Video Editing"	Only Video Editing tasks shown	Pass
TC-14	Apply for task	Bid 500, 3 days, proposal text submitted	Application stored; appears in My Applications	Pass
TC-15	Duplicate application prevention	Apply to same task twice	Error: "Already applied"	Pass
TC-16	Submit work after assignment	Work URL + note submitted	Submission stored; employer can view	Pass
TC-17	Profile edit	Update name, skills, bio	Database updated; profile page shows new data	Pass
TC-18	Leaderboard loads	Open leaderboard.php	Top 20 candidates with ratings displayed	Pass

Test Cases – Employer Module

TC ID	Test Case	Input	Expected Output	Result
TC-19	Post task with valid data	Title, category, price > 0, description, deadline	Task created; appears in Manage Tasks	Pass
TC-20	Post task with price = 0	Price = 0	Error: "Price must be greater than 0."	Pass
TC-21	View applicants	Open view_applicants.php for task with applications	All proposals with bid, days, proposal shown	Pass
TC-22	Assign task	Click "Assign" for one applicant	application.is_selected=1; task.status="assigned"	Pass
TC-23	Review and rate	Rating=5, comment submitted	Review stored; task.status="completed"	Pass
TC-24	Duplicate rating prevention	Submit second rating for same task	Error displayed; no duplicate review created	Pass
TC-25	Employer cannot access candidate dashboard	Employer navigates to candidate_dashboard.php	Redirected to index.html	Pass

Test Cases – Admin Module

TC ID	Test Case	Input	Expected Output	Result
TC-26	Ban user	Click "Ban" on a candidate	is_banned=1; user cannot login	Pass
TC-27	Unban user	Click "Unban" on a banned user	is_banned=0; user can login again	Pass
TC-28	Delete user	Click "Delete" on a non-admin user	User and CASCADE data removed; no orphans	Pass
TC-29	Delete task	Click "Delete" on a task	Task and CASCADE applications removed	Pass
TC-30	Non-admin cannot access admin dashboard	Candidate navigates to admin_dashboard.php	Redirected to index.html	Pass

Security Testing

Test	Method	Result
SQL Injection – Login	Enter ' OR 1=1 -- in email field	Prepared statement neutralises; login fails
XSS – Task Title	Post task with title <script>alert(1)</script>	htmlspecialchars() escapes output; no script executes
CSRF – Admin Ban	Direct URL access: admin_dashboard.php?ban=1	Session check passes only if admin is logged in
Session Fixation	Attempt to reuse session ID before login	session_regenerate_id() invalidates old session
Password Storage	Inspect database password column	Bcrypt hash ($2y$10$...) stored; not plaintext
Unauthorised Role Access	Candidate accesses employer_dashboard.php	Role check fails; redirect to index.html

FUTURE ENHANCEMENT

Short-Term Enhancements (3–6 months)

1. In-Platform Messaging: Add a real-time or asynchronous messaging module allowing candidates and employers to communicate within the platform, reducing reliance on external WhatsApp/email communication.
2. Email Notification System: Implement automated email notifications via PHPMailer for key events — new application received, task assigned, work submitted, review posted — using SMTP.
3. File Upload for Submissions: Allow candidates to upload work files (PDFs, images, ZIP) directly to the server rather than providing external links, improving submission traceability.
4. Advanced Profile: Add profile photo upload, portfolio image gallery (using the existing portfolio_items table), and a dedicated "Skills" tag system with pre-defined tags.
5. Task Bookmarking: Allow candidates to save (bookmark) interesting tasks for later review without immediately applying.

13.2 Medium-Term Enhancements (6–12 months)

6. Payment Gateway Integration: Integrate Razorpay or PayTM for in-platform escrow payments — employer deposits the task budget; funds are released to the candidate upon employer approval of submitted work.
7. Mobile Application: Develop native iOS and Android apps (React Native or Flutter) using the platform's PHP backend as a REST API (JSON responses).
8. Advanced Search and Filters: Add multi-category selection, price range slider, deadline range, and location filter to the task browsing experience.
9. Two-Way Rating: Allow candidates to rate employers after task completion, creating mutual accountability and helping candidates avoid unreliable employers.
10. Dispute Resolution System: Implement a formal dispute mechanism where either party can escalate a task to admin review if a disagreement arises over submission quality or payment.

13.3 Long-Term Enhancements (12+ months)

11. AI-Powered Task Matching: Use machine learning to match candidates with tasks based on their skills, past performance, and task requirements — reducing manual browsing effort.

12. Analytics Dashboard: Provide employers with data visualizations showing applicant quality trends, average bid amounts per category, and time-to-assignment metrics.
13. Institution Partnerships: Enable universities to white-label the platform for their student community, with institution-specific task categories, bulk user registration, and academic credit integration.
14. Multi-Language Support: Add internationalisation (i18n) to support regional Indian languages (Hindi, Gujarati, Tamil) for broader student accessibility.
15. Blockchain Credential Verification: Issue tamper-proof digital certificates (NFT-based or blockchain-anchored) for completed tasks, providing verifiable work history that students can share with future employers.

13.4 Technical Debt and Improvements

- Migrate from MySQLi to PDO for a more portable, consistent database abstraction layer.
- Implement MVC (Model-View-Controller) architecture using a lightweight framework (Laravel or CodeIgniter) to separate business logic from view rendering.
- Add CSRF token validation to all POST forms to prevent cross-site request forgery.
- Implement rate limiting on the login endpoint to prevent brute-force password attacks.
- Add comprehensive server-side input validation with a validation library rather than ad-hoc checks.

BIBLIOGRAPHY/ REFERENCES

Books

1. Nixon, R. (2021). Learning PHP, MySQL & JavaScript (6th ed.). O'Reilly Media.
2. Welling, L. & Thomson, L. (2017). PHP and MySQL Web Development (5th ed.). Addison-Wesley Professional.
3. DuBois, P. (2013). MySQL Cookbook (3rd ed.). O'Reilly Media.
4. Frisbie, M. (2022). JavaScript: The Definitive Guide (7th ed.). O'Reilly Media.

14.2 Online Documentation

5. PHP Documentation. (n.d.). PHP Manual. Retrieved from https://www.php.net/manual/en/
6. MySQL. (n.d.). MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/
7. MDN Web Docs. (n.d.). HTML5 Reference. Retrieved from https://developer.mozilla.org/en-US/docs/Web/HTML
8. MDN Web Docs. (n.d.). CSS Reference. Retrieved from https://developer.mozilla.org/en-US/docs/Web/CSS
9. Chart.js. (n.d.). Chart.js Documentation. Retrieved from https://www.chartjs.org/docs/latest/
10. Font Awesome. (n.d.). Font Awesome Icons Documentation. Retrieved from https://fontawesome.com/docs

14.3 Web References

11. OWASP Foundation. (2021). OWASP Top Ten Web Application Security Risks. Retrieved from https://owasp.org/www-project-top-ten/
12. W3Schools. (n.d.). PHP Tutorial. Retrieved from https://www.w3schools.com/php/
13. StackOverflow Community. (n.d.). PHP and MySQL questions. Retrieved from https://stackoverflow.com/questions/tagged/php+mysql
14. Google Fonts. (n.d.). Inter font family. Retrieved from https://fonts.google.com/specimen/Inter
15. GitHub. (n.d.). docx JavaScript library. Retrieved from https://github.com/dolanmiu/docx

14.4 Academic References

16. Sommerville, I. (2015). Software Engineering (10th ed.). Pearson Education.

17. Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.

18. Connolly, T. & Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation, and Management (6th ed.). Pearson.